

Projet ETL Pipeline avec MongoDB

COMPTE RENDU

Réalisé par : Fatima zahra CHAHID

Encadré par : Mme. Asmae BENTALEB

Filière : Master GLCC

Table des matières

1.	Introduction	2
2.	Technologies utilisées	2
3.	Architecture du projet	2
4.	Mise en place de l'environnement.....	3
5.	Phase Extract	3
5.1	Extraction depuis un fichier CSV.....	3
a.	Lecture du fichier CSV	4
5.2	Extraction depuis SQLite	4
a.	Code de création de la table clients	5
b.	Lecture du table clients	6
5.3	Extraction depuis une API.....	6
a.	Code de l'API Open-Meteo.....	7
6.	Phase Transform.....	8
a.	Suppression des doublons	8
b.	Gestion des valeurs manquantes	9
c.	Uniformisation des données.....	10
7.	Phase Load	10
7.1	Collection utilisée.....	10
7.2	Chargement des données	11
a.	Chargement des données CSV dans MongoDB	11
b.	Chargement des données SQLile dans MongoDB	13
b.	Code de l'API Open-Meteo.....	15
8.	Exécution globale du pipeline ETL	16
8.1	Intégration des différentes étapes dans main.py.....	16
a.	Code du fichier main.py.....	17
8.2	Exécution du pipeline ETL.....	19
9.	Journalisation des exécutions.....	19
9.1	Gestion des journaux d'exécution.....	19
9.2	Cas de succès.....	19
9.3	Gestion des erreurs	20

1. Introduction

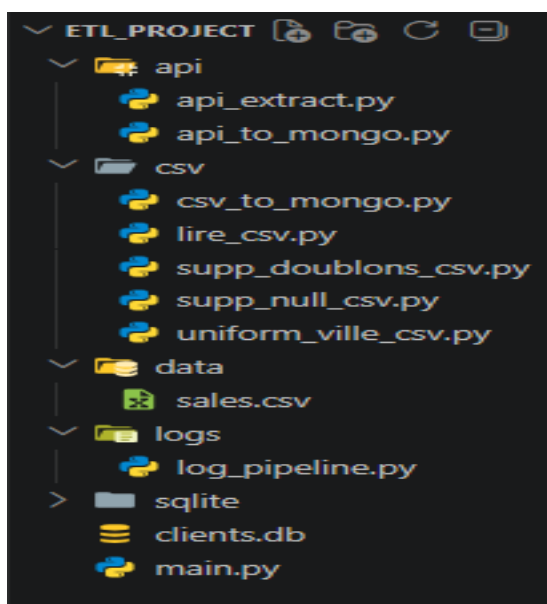
L'objectif de ce projet est de développer un pipeline ETL (Extract, Transform, Load) permettant d'extraire des données provenant de plusieurs sources, de les transformer afin d'améliorer leur qualité puis de les charger dans une base de données MongoDB.

Le pipeline exploite trois sources de données différentes : un fichier CSV, une base de données SQLite et une API météo.

2. Technologies utilisées

Technologie	Description
Python	Langage de programmation principal
Pandas	Manipulation et transformation des données
SQLite	Base de données relationnelle
MongoDB	Base NoSQL utilisée pour le stockage final
PyMongo	Communication avec MongoDB
Requests	Consommation des APIs REST
MongoDB Compass	Interface graphique MongoDB
VS Code	Environnement de développement

3. Architecture du projet



1. Mise en place de l'environnement

Installation des bibliothèques Python :

```
Administrateur : Invite de commandes - pip install pandas pymongo requests schedule sqlalchemy
Microsoft Windows [version 10.0.26200.8655]
(c) Microsoft Corporation. Tous droits réservés.

C:\Windows\System32>pip install pandas pymongo requests schedule sqlalchemy
Collecting pandas
  Downloading pandas-2.3.3-cp310-cp310-win_amd64.whl (11.3 MB)
    | 11.3 MB 1.1 MB/s
Collecting pymongo
  Downloading pymongo-4.17.0-cp310-cp310-win_amd64.whl (815 kB)
    | 815 kB ...
Collecting requests
  Downloading requests-2.34.2-py3-none-any.whl (73 kB)
    | 73 kB 697 kB/s
Collecting schedule
  Downloading schedule-1.2.2-py3-none-any.whl (12 kB)
Collecting sqlalchemy
  Downloading sqlalchemy-2.0.51-cp310-cp310-win_amd64.whl (2.1 MB)
    | 2.1 MB 6.8 MB/s
Collecting python-dateutil>=2.8.2
  Downloading python_dateutil-2.9.0.post0-py2.py3-none-any.whl (229 kB)
    | 229 kB 6.4 MB/s
Collecting tzdata>=2022.7
  Downloading tzdata-2026.3-py2.py3-none-any.whl (348 kB)
    | 348 kB ...
Collecting pytz>=2020.1
  Downloading pytz-2026.2-py2.py3-none-any.whl (510 kB)
    | 510 kB 6.4 MB/s
```

2. Phase Extract

Dans ce projet, les données sont extraites depuis trois sources différentes :

- Un fichier CSV
- Une base de données SQLite
- Une API REST (Open-Meteo)

2.1 Extraction depuis un fichier CSV

Le fichier **sales.csv** contient les informations des ventes.

```
data > sales.csv > data
1 id,product,price,city
2 1,Laptop,5000,Rabat
3 2,Mouse,100,Casablanca
4 3,Keyboard,250,Tanger
5 4,Screen,2000,Kenitra
6 4,Screen,2000,Kenitra
7 5,Imprimante,8000,
```

a. Lecture du fichier CSV

```
csv >  lire_csv.py > ...  
1 # Importation de la bibliothèque Pandas  
2 import pandas as pd  
3  
4 # Lecture du fichier CSV  
5 df = pd.read_csv("data/sales.csv")  
6  
7 # Affichage des données extraites  
8 print("Contenu du fichier CSV :")  
9 print(df)
```

Exécution dans le terminal :

```
PS D:\master_cours\nosql\tp nosql\ETL_Project> python csv\lire_csv.py  
Contenu du fichier CSV :  
   id  product  price  city  
0   1   Laptop   5000  Rabat  
1   2   Mouse    100 Casablanca  
2   3  Keyboard    250  Tanger  
3   4   Screen  2000  Kenitra  
4   4   Screen  2000  Kenitra  
5   5  Imprimante 8000    NaN
```

2.2 Extraction depuis SQLite

Une base SQLite a été créée afin de stocker les informations des clients.

Structure de la table clients

id	nom	prenom	num_tele	ville
----	-----	--------	----------	-------

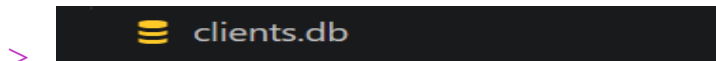
a. Code de création de la table clients

```
sqlite > create_db.py >...
1 # Importation de la bibliothèque SQLite
2 import sqlite3
3
4 # Création ou ouverture de la base de données
5 conn = sqlite3.connect("clients.db")
6
7 # Création d'un curseur
8 cursor = conn.cursor()
9
10 # Création de la table clients
11 cursor.execute("""
12 CREATE TABLE IF NOT EXISTS clients(id INTEGER PRIMARY KEY, nom VARCHAR(50), prenom VARCHAR(50), num_tele INTEGER, ville VARCHAR(50))
13 """)
14
15 # Insertion des données
16 cursor.execute("""
17 INSERT INTO clients(nom, prenom, num_tele, ville) VALUES ('Ahmed', 'Alami', 612345678, 'Rabat')
18 """)
19
20 cursor.execute("""
21 INSERT INTO clients(nom, prenom, num_tele, ville) VALUES ('Sara', 'Karimi', 623456789, 'Kenitra')
22 """)
23
24 cursor.execute("""
25 INSERT INTO clients(nom, prenom, num_tele, ville) VALUES ('Youssef', 'Idrissi', 634567890, 'Casablanca')
26 """)
27
28 # Validation des modifications
29 conn.commit()
30
31 # Fermeture de la connexion
32 conn.close()
33
34 print("Base SQLite créée avec succès")
35
```

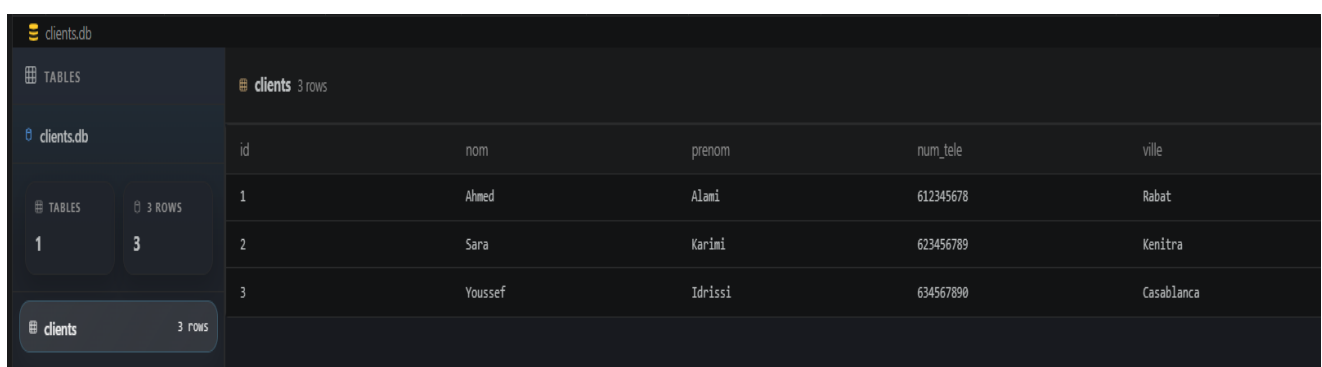
Exécution dans le terminal :

```
PS D:\master_cours\nosql\tp nosql\ETL_Project> python sqlite\create_db.py
Base SQLite créée avec succès
```

On observe que le fichier **clients.db** a été créé avec succès



Qui contient :



id	nom	prenom	num_tele	ville
1	Ahmed	Alami	612345678	Rabat
2	Sara	Karimi	623456789	Kenitra
3	Youssef	Idrissi	634567890	Casablanca

b. Lecture du table clients

```
sqlite > sqlite_extract.py > ...
1  # Importation des bibliothèques
2  import sqlite3
3  import pandas as pd
4
5  # Connexion à la base SQLite
6  conn = sqlite3.connect("clients.db")
7
8  # Lecture des données
9  df = pd.read_sql_query(
10     "SELECT * FROM clients",
11     conn
12 )
13
14 # Affichage du contenu de la table
15 print(df)
16
17 # Fermeture de la connexion
18 conn.close()
19
```

Exécution dans le terminal :

```
PS D:\master_cours\nosql\tp nosql\ETL_Project> python sqlite\sqlite_extract.py
   id    nom  prenom  num_tele  ville
0    1  Ahmed   Alami  612345678  Rabat
1    2   Sara  Karimi  623456789  Kenitra
2    3 Youssef  Idrissi  634567890  Casablanca
```

2.3 Extraction depuis une API

Les données météorologiques sont récupérées depuis l'API Open-Meteo pour la ville de Kénitra.

Les informations récupérées sont :

- Ville
- Température
- Vitesse du vent

a. Code de l'API Open-Meteo

```
api > api_extract.py > ...
1 # Importation des bibliothèques nécessaires
2 import requests
3 import pandas as pd
4
5 # URL de l'API Open-Meteo (ville de Kénitra)
6 url = "https://api.open-meteo.com/v1/forecast?latitude=34.2610&longitude=-6.5802&current=temperature_2m,wind_speed_10m"
7
8 # Envoi de la requête HTTP
9 response = requests.get(url)
10
11 # Conversion de la réponse JSON
12 data = response.json()
13
14 # Création d'un dictionnaire contenant les données météo
15 meteo = {
16     "ville": "Kenitra",
17     "temperature": data["current"]["temperature_2m"],
18     "vitesse_vent": data["current"]["wind_speed_10m"]
19 }
20
21 # Conversion en DataFrame
22 df = pd.DataFrame([meteo])
23
24 # Affichage des données extraites
25 print(df)
```

Exécution dans le terminal :

```
PS D:\master_cours\nosql\tp nosql\ETL_Project> python api\api_extract.py
   ville  temperature  vitesse_vent
0 Kenitra         24.8          13.3
```


3. Phase Transform

a. Suppression des doublons

Les doublons présents dans le fichier **sales.csv** sont supprimés afin d'éviter les redondances.

```
csv > 🐍 supp_doublons_csv.py > ...
1  # Importation de la bibliothèque Pandas
2  import pandas as pd
3
4  # Lecture du fichier CSV
5  df = pd.read_csv("data/sales.csv")
6
7  print("Avant suppression des doublons :")
8  print(df)
9
10 # Suppression des doublons
11 df = df.drop_duplicates()
12
13 print("\nAprès suppression des doublons :")
14 print(df)
```

Exécution dans le terminal :

```
PS D:\master_cours\nosql\tp nosql\ETL_Project> python csv\supp_doublons_csv.py
Avant suppression des doublons :
   id  product  price  city
0   1   Laptop  5000   Rabat
1   2    Mouse   100 Casablanca
2   3  Keyboard   250   Tanger
3   4   Screen  2000   Kenitra
4   4   Screen  2000   Kenitra

Après suppression des doublons :
   id  product  price  city
0   1   Laptop  5000   Rabat
1   2    Mouse   100 Casablanca
2   3  Keyboard   250   Tanger
3   4   Screen  2000   Kenitra
PS D:\master_cours\nosql\tp nosql\ETL_Project> █
```

b. Gestion des valeurs manquantes

Les lignes contenant des valeurs nulles sont supprimées.

```
csv > 🐍 supp_null_csv.py > ...
1 # Importation de la bibliothèque Pandas
2 import pandas as pd
3
4 # Lecture du fichier CSV
5 df = pd.read_csv("data/sales.csv")
6
7 print("Avant suppression des valeurs manquantes :")
8 print(df)
9
10 # Suppression des lignes contenant des valeurs nulles
11 df = df.dropna()
12
13 print("\nAprès suppression des valeurs manquantes :")
14 print(df)
```

Exécution dans le terminal :

```
PS D:\master_cours\nosql\tp nosql\ETL_Project> python csv\supp_null_csv.py
Avant suppression des valeurs manquantes :
   id  product  price  city
0  1  Laptop   5000   Rabat
1  2   Mouse    100 Casablanca
2  3  Keyboard   250    NaN
3  4   Screen   2000  Kenitra

Après suppression des valeurs manquantes :
   id  product  price  city
0  1  Laptop   5000   Rabat
1  2   Mouse    100 Casablanca
3  4   Screen   2000  Kenitra
PS D:\master_cours\nosql\tp nosql\ETL_Project> █
```

c. Uniformisation des données

Les noms des **villes** sont convertis en **majuscules** afin d'assurer l'uniformité des données.

```
csv > python uniform_ville_csv.py > ...
1  # Importation de la bibliothèque Pandas
2  import pandas as pd
3
4  # Lecture du fichier CSV
5  df = pd.read_csv("data/sales.csv")
6
7  print("Avant uniformisation :")
8  print(df)
9
10 # Conversion des noms des villes en majuscules
11 df["city"] = df["city"].str.upper()
12
13 print("\nAprès uniformisation :")
14 print(df)
15
```

Exécution dans le terminal :

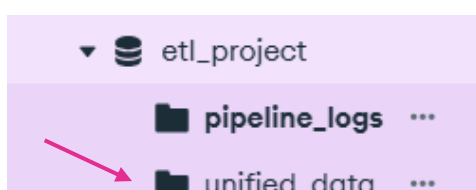
```
PS D:\master_cours\nosql\tp nosql\ETL_Project> python csv\uniform_ville_csv.py
Avant uniformisation :
   id  product  price    city
0   1   Laptop   5000    Rabat
1   2    Mouse    100 Casablanca
2   3  Keyboard    250    Tanger
3   4   Screen   2000   Kenitra

Après uniformisation :
   id  product  price    city
0   1   Laptop   5000    RABAT
1   2    Mouse    100 CASABLANCA
2   3  Keyboard    250    TANGER
3   4   Screen   2000   KENITRA
```

4. Phase Load

Après transformation, les données sont chargées dans MongoDB.

4.1 Collection utilisée



4.2 Chargement des données

Les données provenant des trois sources sont converties en documents JSON puis insérées dans la collection **unified_data**.

a. Chargement des données CSV dans MongoDB

```
csv > 🐍 csv_to_mongo.py > ...
1  # Importation des bibliothèques nécessaires
2  import pandas as pd
3  from pymongo import MongoClient
4
5  # Lecture du fichier CSV
6  df = pd.read_csv("data/sales.csv")
7
8  # Connexion à MongoDB
9  client = MongoClient("mongodb://localhost:27017/")
10
11 # Sélection de la base de données
12 db = client["etl_project"]
13
14 # Sélection de la collection
15 collection = db["unified_data"]
16
17 # Conversion du DataFrame en liste de dictionnaires
18 data = df.to_dict("records")
19
20 # Insertion des données dans MongoDB
21 collection.insert_many(data)
22
23 # Message de confirmation
24 print("Données CSV insérées avec succès dans MongoDB")
```

Exécution dans le terminal :

```
PS D:\master_cours\nosql\tp nosql\ETL_Project> python csv\csv_to_mongo.py
Données CSV insérées avec succès dans MongoDB
```

La figure suivante présente les données insérées et stockées dans la base de données MongoDB.

```
_id: ObjectId('6a564950228ee5b1d5d0027f')  
id: 1  
product: "Laptop"  
price: 5000  
city: "Rabat"
```

```
_id: ObjectId('6a564950228ee5b1d5d00280')  
id: 2  
product: "Mouse"  
price: 100  
city: "Casablanca"
```

```
_id: ObjectId('6a564950228ee5b1d5d00281')  
id: 3  
product: "Keyboard"  
price: 250  
city: "Tanger"
```

```
_id: ObjectId('6a564950228ee5b1d5d00282')  
id: 4  
product: "Screen"  
price: 2000  
city: "Kenitra"
```

```
_id: ObjectId('6a564950228ee5b1d5d00283')
```

b. Chargement des données SQLite dans MongoDB

Les données extraites depuis SQLite sont ensuite converties en documents **JSON** puis insérées dans la collection **unified_data**.

```
sqlite > sqlite_to_mongo.py > ...
1  # Importation des bibliothèques nécessaires
2  import sqlite3
3  import pandas as pd
4  from pymongo import MongoClient
5
6  # Connexion à la base de données SQLite
7  conn = sqlite3.connect("clients.db")
8
9  # Lecture des données de la table clients
10 df = pd.read_sql_query(
11     "SELECT * FROM clients",
12     conn
13 )
14
15 # Affichage des données récupérées
16 print(df)
17
18 # Connexion au serveur MongoDB
19 client = MongoClient("mongodb://localhost:27017/")
20
21 # Sélection de la base de données
22 db = client["etl_project"]
23
24 # Sélection de la collection
25 collection = db["unified_data"]
26
27 # Conversion du DataFrame en liste de dictionnaires
28 data = df.to_dict("records")
29
30 # Insertion des données dans MongoDB
31 collection.insert_many(data)
32
33 print("Les données SQLite ont été insérées dans MongoDB avec succès.")
34
35 # Fermeture de la connexion SQLite
36 conn.close()
```

Exécution dans le terminal :

```
PS D:\master_cours\nosql\tp nosql\ETL_Project> python sqlite\sqlite_to_mongo.py
Les données SQLite ont été insérées dans MongoDB avec succès.
```

La figure suivante présente les données des clients insérées et stockées dans la base de données MongoDB.

```
_id: ObjectId('6a54f70ddb6577825424f31f')  
id : 1  
nom : "Ahmed"  
prenom : "Alami"  
num_tele : 612345678  
ville : "Rabat"
```

```
_id: ObjectId('6a54f70ddb6577825424f320')  
id : 2  
nom : "Sara"  
prenom : "Karimi"  
num_tele : 623456789  
ville : "Kenitra"
```

```
_id: ObjectId('6a54f70ddb6577825424f321')  
id : 3  
nom : "Youssef"  
prenom : "Idrissi"  
num_tele : 634567890  
ville : "Casablanca"
```

b. Code de l'API Open-Meteo

```
api > api_to_mongo.py > ...
1 # Importation des bibliothèques nécessaires
2 import requests
3 import pandas as pd
4 from pymongo import MongoClient
5
6 # URL de l'API Open-Meteo
7 url = "https://api.open-meteo.com/v1/forecast?latitude=34.2610&longitude=-6.5802&current=temperature_2m,wind_speed_10m"
8
9 # Récupération des données
10 response = requests.get(url)
11 data = response.json()
12
13 # Création du dictionnaire
14 meteo = {
15     "ville": "Kenitra",
16     "temperature": data["current"]["temperature_2m"],
17     "vitesse_vent": data["current"]["wind_speed_10m"]
18 }
19
20 # Conversion en DataFrame
21 df = pd.DataFrame([meteo])
22
23 # Connexion à MongoDB
24 client = MongoClient("mongodb://localhost:27017/")
25
26 db = client["etl_project"]
27
28 collection = db["unified_data"]
29
30 # Insertion dans MongoDB
31 collection.insert_many(df.to_dict("records"))
32
33 print("Données météo insérées avec succès dans MongoDB")
```

Exécution dans le terminal :

```
PS D:\master_cours\nosql\tp nosql\ETL_Project> python api\api_extract.py
Les données météo ont été insérées dans MongoDB avec succès.
```

La figure suivante montre les données météo de Kénitra enregistrées dans la base de données MongoDB.

```
_id: ObjectId('6a564dbd80e63324560b9bc7')
ville: "Kenitra"
temperature: 27.5
vitesse_vent: 14
```


5. Exécution globale du pipeline ETL

5.1 Intégration des différentes étapes dans main.py

Après avoir développé et testé séparément les différentes phases d'extraction, de transformation et de chargement, l'ensemble des traitements a été regroupé dans un fichier principal nommé **main.py**.

Ce fichier permet d'automatiser l'exécution complète du pipeline ETL en enchaînant les opérations suivantes :

- Extraction des données depuis le fichier CSV.
- Extraction des données depuis la base de données SQLite.
- Extraction des données depuis l'API Open-Meteo.
- Transformation des données (suppression des doublons, suppression des valeurs manquantes et uniformisation des noms des villes).
- Chargement des données consolidées dans la collection MongoDB **unified_data**.
- Enregistrement du statut d'exécution dans la collection **pipeline_logs**.

a. Code du fichier main.py

```
main.py > ...
1  # Importation des bibliothèques nécessaires
2  import pandas as pd
3  import sqlite3
4  import requests
5  from pymongo import MongoClient
6  from datetime import datetime
7
8  # Connexion à MongoDB
9  client = MongoClient("mongodb://localhost:27017/")
10 db = client["etl_project"]
11
12 # Collections MongoDB
13 collection = db["unified_data"]
14 logs = db["pipeline_logs"]
15
16 try:
17
18     # Vider la collection avant chaque exécution
19     collection.delete_many({})
20
21     print("Anciennes données supprimées.")
22
23     # =====
24     # Extraction depuis le fichier CSV
25     # =====
26
27     df_csv = pd.read_csv("data/sales.csv")
28
29     # Transformation des données
30     df_csv = df_csv.drop_duplicates()
31     df_csv = df_csv.dropna()
32     df_csv["city"] = df_csv["city"].str.upper()
33
34     # Chargement dans MongoDB
35     collection.insert_many(df_csv.to_dict("records"))
36
37     print("Données CSV chargées avec succès.")
38
```

```

39     # =====
40     # Extraction depuis SQLite
41     # =====
42
43     conn = sqlite3.connect("clients.db")
44
45     df_sqlite = pd.read_sql_query(
46         "SELECT * FROM clients",
47         conn
48     )
49
50     collection.insert_many(df_sqlite.to_dict("records"))
51
52     conn.close()
53
54     print("Données SQLite chargées avec succès.")
55
56     # =====
57     # Extraction depuis l'API Open-Meteo
58     # =====
59
60     url = "https://api.open-meteo.com/v1/forecast?latitude=34.2610&longit
61
62     response = requests.get(url)
63     data = response.json()
64
65     meteo = {
66         "ville": "Kenitra",
67         "temperature": data["current"]["temperature_2m"],
68         "vitesse_vent": data["current"]["wind_speed_10m"]
69     }
70
71     collection.insert_one(meteo)
72
73     print("Données API chargées avec succès.")
74
75     # =====
76     # Enregistrement du journal
77     # =====
78
79     logs.insert_one({
80         "date_execution": datetime.now(),
81         "status": "Succès",
82         "message": "Pipeline ETL exécuté avec succès"
83     })
84
85     print("Pipeline ETL terminé avec succès.")
86
87 except Exception as e:
88
89     logs.insert_one({
90         "date_execution": datetime.now(),
91         "status": "Erreur",
92         "message": str(e)
93     })
94
95     print("Erreur :", e)
96

```

5.2 Exécution du pipeline ETL

L'exécution du pipeline est réalisée à l'aide de la commande suivante :

```
PS D:\master_cours\nosql\tp nosql\ETL_Project> python main.py
Anciennes données supprimées.
Données CSV chargées avec succès.
Données SQLite chargées avec succès.
Données API chargées avec succès.
Pipeline ETL terminé avec succès.
```

6. Journalisation des exécutions

6.1 Gestion des journaux d'exécution

Afin d'assurer le suivi du pipeline ETL, une collection MongoDB nommée `pipeline_logs` a été créée.

Cette collection permet d'enregistrer automatiquement les informations relatives à chaque exécution du pipeline :

- Date et heure d'exécution.
- Statut de l'exécution (Succès ou Erreur).
- Message descriptif associé à l'exécution.

6.2 Cas de succès

Lorsque toutes les étapes du pipeline s'exécutent correctement, un document de journalisation est enregistré avec le statut **Succès**.

```
_id: ObjectId('6a5788ced956643b38ac0fe1')
date_execution : 2026-07-15T14:19:10.551+00:00
status : "Succès"
message : "Pipeline ETL exécuté avec succès"
```

6.3 Gestion des erreurs

Le fichier **main.py** utilise un bloc **try/except** permettant de capturer les exceptions pouvant survenir durant l'exécution du pipeline.

- En cas d'erreur (fichier introuvable, problème de connexion ou erreur SQL), un document est automatiquement enregistré dans la collection **pipeline_logs** avec le statut **Erreur**.

```
_id: ObjectId('6a578b872bc86333473a2908')
date_execution : 2026-07-15T14:30:47.137+00:00
status : "Erreur"
message : "Execution failed on sql 'SELECT * FROM clientssss': no such table: cli..."
```

FIN
